K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

Fundamentals Of Operating System Lab

Lab Assignment

ENCA252

For

BCA(AI & DS)

(2026-27)

Prepared by : Deep Sharma **Submitted To : Dr. Sneha Yadav**

Roll No : 2401201211

Problem Title:

Implementation and Analysis of CPU Scheduling Algorithms (FCFS & SJF)

Problem Statement

CPU Scheduling is a core function of an operating system that determines the order in which processes access the CPU. In this assignment, students will implement FCFS and SJF scheduling algorithms and evaluate their performance using standard metrics.

Learning Objectives

- Understand CPU scheduling concepts
- Implement FCFS and SJF algorithms
- Calculate CT, TAT, WT
- Analyze system performance **Tools/Technology Used:**
- Python 3.x
- Built-in Python libraries: os, multiprocessing, time
- Linux OS (optional for simulation)

Assignment Tasks:

Task 1: Process Creation & Input Handling

Objective:

To understand how processes are represented in an operating system and how input data is structured for scheduling.

Description:

In an operating system, each process has attributes such as Process ID (PID), Arrival Time (AT), and Burst Time (BT). This task focuses on creating a process model using a class and taking user input.

Sub-Tasks:

1. Create a class Process with attributes:

- Process ID (PID)
- Arrival Time (AT)
- Burst Time (BT)

2. Accept input for at least 4–5 processes from the user.
3. Store processes in a list or array.
4. Display the input in tabular format.

Expected Outcome:

- Students will understand process structure.
- Proper input handling and data storage.

Input :

```
# -------- Task 1: Process Class --------
class Process:
    def __init__(self, pid, at, bt):
        self.pid = pid
        self.at = at
        self.bt = bt
        (parameter) self: Self@Process
        self.tat = 0
        self.wt = 0


# -------- Input Function --------
def take_input():
    n = int(input("Enter number of processes: "))
    processes = []

    for i in range(n):
        at = int(input(f"Arrival Time of P{i+1}: "))
        bt = int(input(f"Burst Time of P{i+1}: "))
        processes.append(Process(i+1, at, bt))

    print("\nPID\tAT\tBT")
    for p in processes:
        print(f"P{p.pid}\t{p.at}\t{p.bt}")

    return processes
```

Output :

```
Enter number of processes: 4
Arrival Time of P1: 6
Burst Time of P1: 3
Arrival Time of P2: 8
Burst Time of P2: 4
Arrival Time of P3: 10
Burst Time of P3: 5
Arrival Time of P4: 12
Burst Time of P4: 6

PID     AT      BT
P1      6       3
P2      8       4
P3      10      5
P4      12      6
```

Task 2: FCFS Scheduling Implementation

Objective:

To implement the First Come First Serve (FCFS) scheduling algorithm.

Description:

FCFS is the simplest scheduling algorithm where the process that arrives first gets executed first. It is non-preemptive in nature.

Sub-Tasks:

1. Sort processes based on Arrival Time.
2. Calculate:
   - Completion Time (CT) o Turnaround Time (TAT) = CT – AT o Waiting Time (WT) = TAT – BT
3. Handle CPU idle condition (if no process has arrived yet).
4. Display results in a table.

Expected Outcome:

- Understanding of FCFS working.
- Ability to compute scheduling parameters.

Input:

```python
# -------- Task 2: FCFS --------
def fcfs(processes):
    pro = sorted(processes, key=lambda x: x.at)
    time = 0

    print("\n--- FCFS ---")
    print("PID\tAT\tBT\tCT\tTAT\tWT")

    total_wt = total_tat = 0

    for p in pro:
        if time < p.at:
            time = p.at

        time += p.bt
        p.ct = time
        p.tat = p.ct - p.at
        p.wt = p.tat - p.bt

        total_wt += p.wt
        total_tat += p.tat

        print(f"P{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")

    print("Average WT =", total_wt/len(pro))
    print("Average TAT =", total_tat/len(pro))

    return pro
```

Output :

```
--- FCFS ---
PID     AT      BT      CT      TAT     WT
P1      6       3       9       3       0
P2      8       4       13      5       1
P3      10      5       18      8       3
P4      12      6       24      12      6
Average WT = 2.5
Average TAT = 7.0
```

Task 3: SJF Scheduling (Non-Preemptive)

Objective:

To implement Shortest Job First scheduling.

Description:

SJF selects the process with the smallest burst time from the ready queue. It improves average waiting time but requires careful selection.

Sub-Tasks:

1. From available processes, pick the one with the shortest burst time.

2. Ensure only arrived processes are considered.
3. Calculate:
    - Completion Time (CT)
        - Turnaround Time (TAT)
    - Waiting Time (WT)
4. Continue until all processes are executed.

Expected Outcome:

- Better understanding of optimization in scheduling.
- Comparison with FCFS.

Input :

```python
# -------- Task 3: SJF --------
def sjf(processes):
    pro = processes[:]
    completed = []
    time = 0

    print("\n--- SJF ---")
    print("PID\tAT\tBT\tCT\tTAT\tWT")

    total_wt = total_tat = 0

    while pro:
        ready = [p for p in pro if p.at <= time]

        if not ready:
            time += 1
            continue

        ready.sort(key=lambda x: x.bt)
        p = ready[0]

        time += p.bt
        p.ct = time
        p.tat = p.ct - p.at
        p.wt = p.tat - p.bt

        total_wt += p.wt
        total_tat += p.tat

        print(f"P{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")

        completed.append(p)
        pro.remove(p)
```

```python
print("Average WT =", total_wt/len(completed))
print("Average TAT =", total_tat/len(completed))

return completed
```

Output :

```
--- SJF ---
PID     AT      BT      CT      TAT     WT
P1      6       3       9       3       0
P2      8       4       13      5       1
P3      10      5       18      8       3
P4      12      6       24      12      6
Average WT = 2.5
Average TAT = 7.0
```

Task 4: Gantt Chart Representation

Objective:

To visualize process execution sequence.

Description:

A Gantt chart shows how processes are executed over time.

Sub-Tasks:

1. Draw execution order for:
   - FCFS
   - SJF
2. Represent time intervals clearly.
3. Label processes on timeline.

Expected Outcome:

- Visual understanding of scheduling.
- Improved conceptual clarity.

Input :

```python
# -------- Task 4: Gantt Chart --------
def gantt(process_list, name):
    print(f"\nGantt Chart ({name}):")
    for p in process_list:
        print(f"| P{p.pid} ", end="")
    print("|")
```

Output :

```
Gantt Chart (FCFS):
| P1 | P2 | P3 | P4 |
```

```
Gantt Chart (SJF):
| P1 | P2 | P3 | P4 |
```

Task 5: Performance Analysis & Comparison

Objective:

To evaluate and compare scheduling algorithms.

Description:

Performance is measured using average waiting time and turnaround time.

Sub-Tasks:

1. Calculate:
   - Average Waiting Time
   - Average Turnaround Time
2. Compare FCFS and SJF results.
3. Answer:
   - Which algorithm is better?
   - Why?

Expected Outcome:

- Analytical thinking.
- Understanding trade-offs between algorithms.

Input :

```python
# -------- Task 5: Main --------
if __name__ == "__main__":
    processes = take_input()

    fcfs_result = fcfs(processes)
    gantt(fcfs_result, "FCFS")

    sjf_result = sjf(processes)
    gantt(sjf_result, "SJF")

    print("\n--- Comparison ---")
    fcfs_wt = sum(p.wt for p in fcfs_result) / len(fcfs_result)
    sjf_wt = sum(p.wt for p in sjf_result) / len(sjf_result)

    if sjf_wt < fcfs_wt:
        print("SJF is better (less waiting time)")
    else:
        print("FCFS is better")
```

Output :

```
--- Comparison ---
FCFS is better
PS C:\Users\Lenovo\OneDrive\Documents\Operating System>
```